WEEK – 14

Experiment No. 14: Arrays – Valid Parentheses

Title

Valid Parentheses Using Java Collection Classes

Objective / Aim

To write a Java program to determine whether a string containing parentheses, curly braces, and square brackets is valid using the Java Collection Framework.

Problem Statement

Write a Java program to determine whether a given string containing only the characters `(`, `)`, `{`, `}`, `[` and `]` is valid.

An input string is valid if it satisfies the following conditions:

1. Every opening bracket must be closed by the same type of bracket.
2. Brackets must be closed in the correct order.
3. An empty string is also considered valid.

Example 1:

Input: `()`

Output: `Valid`

Example 2:

Input: `({)}`

Output: `Not valid`

Concepts Used

Concept	Description
Java Collection Framework	Provides classes and interfaces to store and manipulate groups of objects.
Stack	Follows the Last In, First Out (LIFO) principle.
ArrayDeque	A collection class that can be used as a stack.
`push()`	Adds an element to the top of the stack.
`pop()`	Removes the top element from the stack.
`isEmpty()`	Checks whether the stack is empty.
`charAt()`	Retrieves a character at a specified index in a string.
Conditional Statements	Check whether brackets match correctly.

Classes Used

1. **ArrayDeque** – Used as a stack to store opening brackets.
2. **ValidParentheses** – Contains the logic to check whether the given string has valid parentheses and the `main()` method to execute the program.

Algorithm / Logic

1. Start the program.
2. Read a string containing brackets.
3. Create an empty stack using `ArrayDeque`.
4. Traverse each character of the string.
5. If the character is an opening bracket `(`, `{`, or `[`, push it onto the stack.
6. If the character is a closing bracket `)`, `}`, or `]`, check whether the stack is empty.
7. If the stack is empty when a closing bracket is encountered, the string is invalid.
8. Otherwise, pop the top opening bracket from the stack.
9. Check whether the opening and closing brackets match.
10. If they do not match, the string is invalid.
11. After processing all characters, check whether the stack is empty.
12. If the stack is empty and all brackets matched correctly, display `Valid`.
13. Otherwise, display `Not valid`.
14. Stop the program.

Source Code

File Name: ValidParentheses.java

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class ValidParentheses {

    public static boolean isValid(String s) {

        Deque<Character> stack = new ArrayDeque<>();

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            if (ch == '(' || ch == '{' || ch == '[') {

                stack.push(ch);

            } else if (ch == ')' || ch == '}' || ch == ']') {

                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {

                    return false;
                }
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String input = sc.nextLine();

        if (isValid(input)) {
            System.out.println("Valid");
        } else {
            System.out.println("Not valid");
        }
```

```
        sc.close();
    }
}
```

Explanation of the Program

1. ArrayDeque Class

The `ArrayDeque` class is used to implement a stack. Opening brackets are stored in the stack until their corresponding closing brackets are encountered.

2. Opening Brackets

When the program encounters an opening bracket such as `(`, `{`, or `[`, it pushes the bracket onto the stack using the `push()` method.

3. Closing Brackets

When a closing bracket is encountered, the program checks whether the stack is empty. If it is empty, the closing bracket has no matching opening bracket, so the string is invalid.

Otherwise, the program removes the top opening bracket using `pop()` and checks whether it matches the closing bracket.

4. Final Validation

After processing the entire string, the stack must be empty. If opening brackets remain in the stack, they have not been closed, and the string is invalid.

Compilation and Execution

Step 1: Save the program

Save the source code as `ValidParentheses.java`.

Step 2: Compile the program

Open the terminal in the directory containing the file and execute:

```
javac ValidParentheses.java
```

Step 3: Run the program

Execute:

```
java ValidParentheses
```

Sample Output

Test Case 1: Valid Parentheses

Input:

```
()
```

Output:

```
Enter a string: ()
Valid
```

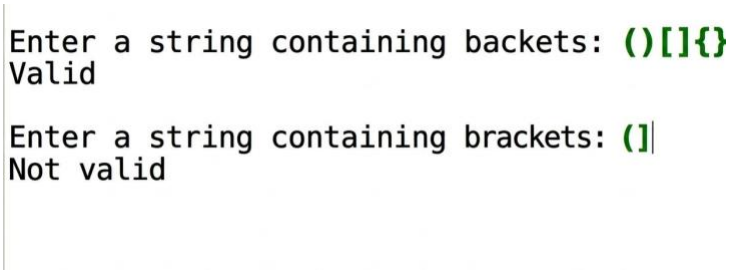
Test Case 2: Invalid Parentheses

Input:

```
({)}
```

Output:

```
Enter a string: ({)}
Not valid
```



Test Cases

Test Case 1: Simple Valid Parentheses

Input: `()`

Expected Output: `Valid`

Explanation: The opening and closing parentheses match correctly.

Test Case 2: Incorrect Order of Brackets

Input: `({)}`

Expected Output: `Not valid`

Explanation: The parentheses close before the inner curly brace is closed.

Test Case 3: Valid Multiple Brackets

Input: `()[]{}`

Expected Output: `Valid`

Explanation: All brackets are correctly matched and closed in the proper order.

Test Case 4: Valid Nested Brackets

Input: `{[()]}`

Expected Output: `Valid`

Explanation: All nested brackets are closed in the correct order.

Test Case 5: Missing Closing Bracket

Input: `((`

Expected Output: `Not valid`

Explanation: The opening parentheses are not closed.

Test Case 6: Extra Closing Bracket

Input: `())`

Expected Output: `Not valid`

Explanation: The final closing parenthesis does not have a corresponding opening parenthesis.

Test Case 7: Empty String

Input: `""` (press Enter without typing any characters)

Expected Output: `Valid`

Explanation: An empty string is considered valid according to the problem statement.

Test Case Summary

Test Case	Input	Expected Output
1	`()`	Valid
2	`({)}`	Not valid
3	`()[]{}`	Valid
4	`{[()]}`	Valid
5	`((`	Not valid
6	`())`	Not valid
7	Empty string	Valid

Result

Thus, the Java program successfully determines whether a given string of parentheses, curly braces, and square brackets is valid by using a stack implemented with the `ArrayDeque` class.

Conclusion

The program demonstrates the use of Java Collection Framework classes to solve the valid parentheses problem. It uses the `ArrayDeque` class as a stack and applies the Last In, First Out (LIFO) principle to check whether brackets are correctly matched and closed in the proper order.